

گزارش تمرین ۳

بابک حسینی محتشم ۸۱۰۱۰۱۴۰۸ - بهراد بینایی ۸۱۰۱۰۱۳۹۲

بخش اول: سوالات تئوری

چرا داده ها در سیستم های توزیع شده replicate می شوند؟

در سیستم های توزیع شده، داده ها معمولاً در چندین replica ذخیره می شوند تا دسترسی پذیری سیستم افزایش یابد و در صورت خرابی یک بخش، سرویس همچنان قابل استفاده باشد. همچنین replication می تواند باعث کاهش زمان پاسخ شود؛ زیرا کاربران می توانند به نزدیک ترین نسخه داده دسترسی پیدا کنند و فشار درخواست ها بین چند سرور تقسیم شود. به طور کلی، مهم ترین دلایل استفاده از replication عبارت اند از: افزایش availability و جلوگیری از از کار افتادن کامل سیستم در صورت خرابی یک سرور. افزایش تحمل خطا tolerance fault. کاهش تاخیر پاسخ دهی با نزدیک تر کردن داده به کاربران. افزایش ظرفیت پردازش درخواست ها با توزیع بار بین چند replica.

تفاوت replication برای افزایش کارایی و replication برای تحمل خطا چیست؟

در replication با هدف افزایش کارایی، هدف اصلی کاهش زمان پاسخ و افزایش ظرفیت سیستم است. در این حالت معمولاً چند نسخه از داده وجود دارد تا درخواست های خواندن بین آن ها تقسیم شود و بار روی یک سرور خاص افزایش پیدا نکند. اما در replication با هدف تحمل خطا، هدف اصلی حفظ عملکرد سیستم هنگام خرابی یک یا چند بخش است. وجود نسخه های اضافی باعث می شود حتی اگر یکی از سرورها از دسترسی خارج شود، نسخه دیگری از داده برای پاسخ گویی وجود داشته باشد. بنابراین تفاوت اصلی این است که در حالت اول تمرکز روی performance و در حالت دوم تمرکز روی reliability و availability است.

مشکل اصلی replication از نظر consistency چیست؟

از آنجا که هر داده در چند replica نگهداری می شود، پس از انجام عملیات نوشتن ممکن است همه نسخه ها در یک لحظه مقدار یکسانی نداشته باشند. برای مثال اگر مقدار یک کلید روی یک replica تغییر کند، ممکن است انتقال این تغییر به سایر replica ها با تاخیر انجام شود. این موضوع باعث ایجاد مشکل read stale می شود؛ یعنی کاربر ممکن است مقدار قدیمی یک داده را از یک replica دریافت کند. بنابراین طراحی سیستم باید بین سرعت انتشار تغییرات و میزان سازگاری داده ها یک تعادل ایجاد کند.

تفاوت strong consistency و eventual consistency چیست؟

در مدل strong consistency پس از انجام موفق عملیات نوشتن، تمام خواندن های بعدی مقدار جدید را مشاهده خواهند کرد. این مدل رفتار سیستم را مشابه یک حافظه واحد و متمرکز نشان می دهد، اما معمولاً هزینه بیشتری از نظر تاخیر و کاهش دسترسی پذیری دارد.

در مدل eventual consistency ممکن است replica ها برای مدت کوتاهی مقادیر متفاوتی داشته باشند، اما اگر عملیات نوشتن جدیدی انجام نشود، در نهایت همه replica ها به یک مقدار مشترک خواهند رسید.

بنابراین consistency strong سازگاری بیشتری ارائه می دهد، در حالی که consistency eventual معمولاً دسترس پذیری و کارایی بالاتری دارد.

مفهوم read-your-writes consistency چیست؟

این مدل یکی از مدل های centric-client consistency است. در این مدل تضمین می شود که اگر یک کاربر مقداری را بنویسد، تمام خواندن های بعدی همان کاربر مقدار جدید نوشته شده را مشاهده خواهند کرد. برای مثال، اگر کاربر مقدار کلید x را از 10 به 20 تغییر دهد، درخواست های بعدی همان کاربر نباید مقدار 10 را مشاهده کنند؛ حتی اگر درخواست خواندن به replica دیگری ارسال شود.

مفهوم monotonic reads چیست؟

در monotonic reads تضمین می شود که مقدار مشاهده شده توسط یک کاربر در طول زمان به عقب بازنگردد. به عبارت دیگر، اگر کاربر در یک لحظه نسخه جدیدتری از یک داده را مشاهده کند، در خواندن های بعدی نباید نسخه قدیمی تر همان داده را دریافت کند. برای مثال، اگر کاربری ابتدا مقدار نسخه ۵ یک کلید را مشاهده کند، مشاهده نسخه ۳ در یک درخواست بعدی نقض monotonic reads خواهد بود.

چرا در سیستم های توزیع شده نمی توان همزمان همیشه بیشترین partition و consistency و availability را به صورت کامل داشت؟

طبق قضیه CAP در سیستم های توزیع شده، هنگام وقوع یک network partition نمی توان هر سه ویژگی Availability، Consistency و Partition Tolerance را به صورت همزمان و کامل تضمین کرد.

منظور از Partition Tolerance توانایی سیستم برای ادامه کار در شرایطی است که ارتباط بین بخش های مختلف سیستم توزیع شده قطع شده یا پیام ها بین آن ها با تاریخ زیاد منتقل می شوند. به عبارت دیگر، اگر شبکه بین دو یا چند replica دچار مشکل شود، سیستم باید بتواند این شرایط را تحمل کند و همچنان عملکرد قابل قبولی داشته باشد.

در یک سیستم دارای چند replica ممکن است به دلیل خرابی شبکه، یک replica نتواند با سایر replica ها ارتباط برقرار کند. در این حالت اگر سیستم بخواهد Consistency کامل را حفظ کند، باید از انجام عملیات روی replica هایی که هماهنگ نیستند جلوگیری کند تا داده های متفاوت ایجاد نشود. این تصمیم باعث کاهش Availability می شود، زیرا بعضی درخواست ها ممکن است با خطا مواجه شوند یا تا زمان برقراری ارتباط مجدد منتظر بمانند.

در مقابل، اگر سیستم بخواهد در زمان network partition همچنان پاسخگو باقی بماند و Availability را حفظ کند، ممکن است هر بخش از سیستم به صورت مستقل عملیات نوشتن را انجام دهد. در نتیجه ممکن است برای مدت کوتاهی مقادیر متفاوتی در replica های مختلف وجود داشته باشد و Consistency کاهش پیدا کند.

بنابراین در هنگام وقوع partition، سیستم های توزیع شده معمولاً مجبور به انتخاب بین حفظ سازگاری بیشتر یا حفظ دسترسی پذیری بیشتر هستند. به همین دلیل بسیاری از سیستم های واقعی از مدل هایی مانند Eventual Consistency استفاده می کنند تا با پذیرش ناسازگاری موقت، دسترسی پذیری بالاتر ارائه دهند.

بخش دوم: پیاده سازی Store Replicated Value-Key

- اجرای حداقل سه Replica مستقل

در این پروژه سه Replica مستقل روی پورت‌های 8081، 8082 و 8083 اجرا شده‌اند که هر کدام دارای فضای ذخیره‌سازی محلی، تنظیمات مستقل و مجموعه‌ای از همتایان (Peers) هستند. هر Replica به صورت یک سرویس HTTP جداگانه اجرا می‌شود و درخواست‌های GET، PUT و Replication را پردازش می‌کند. فایل‌های پیکربندی JSON مشخصات هر Replica شامل شناسه، پورت و لیست همتایان را تعیین می‌کنند.

```

[base] :~/University/Term8/Distributed/HW3/distributed-kv-store$ go run replica/main.go -config=configs/replica1.json
[replica1] Server booting up on port :8081...

[base] :~/University/Term8/Distributed/HW3/distributed-kv-store$ go run replica/main.go -config=configs/replica2.json
[replica2] Server booting up on port :8082...

[base] :~/University/Term8/Distributed/HW3/distributed-kv-store$ go run replica/main.go -config=configs/replica3.json
[replica3] Server booting up on port :8083...

```

• پیاده‌سازی صحیح GET و PUT

عملیات PUT از طریق مسیر `put/` مقدار جدید را در Replica محلی ذخیره کرده و نسخه (Version) رکورد را افزایش می‌دهد. عملیات GET نیز از طریق مسیر `get/` آخرین مقدار ذخیره‌شده برای یک کلید را همراه با نسخه آن باز می‌گرداند. در صورت نبودن کلید یا ارسال درخواست نامعتبر، خطاهای مناسب HTTP به کاربر بازگردانده می‌شود.

```

[replica3] GET key='strong_key_0' -> value='strong_val_0', version=1
[replica3] Replication received key='strong_key_500', value='strong_val_500', version=1 from replica1

```

```

config=configs/replica1.json
[replica1] Server booting up on port :8081...
[replica1] PUT received: key='y', value='eventual_val', mode='eventual', version=1

```

• Replication بین Replicaها

پس از انجام هر عملیات PUT، اطلاعات رکورد شامل کلید، مقدار، نسخه، شناسه Replica نویسنده و زمان ثبت تغییرات برای سایر Replicaها ارسال می‌شود. این فرآیند از طریق مسیر `replicate/` انجام شده و هر Replica پس از دریافت پیام، داده را با وضعیت محلی خود مقایسه کرده و در صورت جدیدتر بودن آن را اعمال می‌کند. همچنین دریافت موفق هر پیام با ارسال ACK ثبت می‌شود.

• پیاده‌سازی مدل Eventual Consistency

در مدل Eventual Consistency، پس از ثبت موفق داده در Replica محلی، عملیات Replication به صورت غیرهمزمان (Asynchronous) در پس‌زمینه انجام می‌شود و پاسخ PUT بلافاصله به کاربر بازگردانده می‌شود. در نتیجه ممکن است برای مدت کوتاهی Replicaهای مختلف مقادیر متفاوتی مشاهده کنند، اما پس از تبادل پیام‌های Replication تمامی Replicaها به یک وضعیت نهایی مشترک همگرا می‌شوند.

• پیاده‌سازی مدل Strong Consistency ساده‌شده

برای پیاده‌سازی Strong Consistency از مکانیزم Quorum استفاده شده است. پس از ثبت مقدار در Replica محلی، پیام Replication به صورت همزمان برای سایر Replicaها ارسال می‌شود و سیستم تنها زمانی عملیات PUT را موفق در نظر می‌گیرد که حداقل اکثریت Replicaها (۲ از ۳ گره) تغییر را تأیید کنند. در صورت عدم دستیابی به Quorum، عملیات ناموفق اعلام شده و تغییر محلی بازگردانده می‌شود. همچنین برای Strong GET، درخواست از Replicaهای ثانویه به Replica اصلی هدایت می‌شود تا جدیدترین مقدار تضمین گردد.

```
~/University/Term8/Distributed/HW3/distributed-kv-store$ go run replica/main.go -config=configs/replica1.json
[replica1] Server booting up on port :8081...
[replica1] PUT received: key='z', value='strong_value', mode='strong', version=1
[replica1] Replication ACK received from http://localhost:8082 for key='z'
[replica1] Replication ACK received from http://localhost:8083 for key='z'
[replica1] STRONG WRITE SUCCESS key='z' quorum=3/3
[replica1] Primary serving strong GET key='z' value='strong_value' version=1

~/University/Term8/Distributed/HW3/distributed-kv-store$ go run replica/main.go -config=configs/replica2.json
[replica2] Server booting up on port :8082...
[replica2] Replication received key='z', value='strong_value', version=1 from replica1
[replica2] Strong GET forwarded to primary for key='z'

~/University/Term8/Distributed/HW3/distributed-kv-store$ go run replica/main.go -config=configs/replica3.json
[replica3] Server booting up on port :8083...
[replica3] Replication received key='z', value='strong_value', version=1 from replica1

~/University/Term8/Distributed/HW3/distributed-kv-store$ go run client/main.go
=====
DISTRIBUTED SYSTEMS SCENARIO ENGINE
=====

>>> Running Strong GET Scenario
[Client] Writing key 'z' = 'strong_value' via Replica 1 (Strong Mode)...
[Client] Performing strong GET for key 'z' from Replica 2...
[STRONG GET] Replica 2 returned value: 'strong_value' with version: 1
(base) ~/University/Term8/Distributed/HW3/distributed-kv-store$
```

```
func runStrongGetScenario() {
    fmt.Println("\n>>> Running Strong GET Scenario")
    setLatency(R1, 1000)
    setLatency(R2, 1000)
    setLatency(R3, 1000)

    fmt.Println("[Client] Writing key 'z' = 'strong_value' via Replica 1 (Strong Mode)...")
    putKey(R1, "z", "strong_value", "strong")

    fmt.Println("[Client] Performing strong GET for key 'z' from Replica 2...")
    val, ver, ok := strongGet(R2, "z")
    if ok {
        fmt.Printf(" [STRONG GET] Replica 2 returned value: '%s' with version: %d\n", val, ver)
    } else {
        fmt.Println(" [STRONG GET] Failed to retrieve value from Replica 2.")
    }
}
```

در این آزمایش مکانیزم Strong Consistency با استفاده از Replica 1 به عنوان Primary پیاده‌سازی شد. ابتدا کلاینت یک عملیات PUT برای کلید z با مقدار strong_value به Replica 1 ارسال کرد. Replica 1 پس از ذخیره محلی داده، پیام تکثیر را برای Replica 2 و Replica 3 ارسال نمود. پس از دریافت تأییدیه از اکثریت Replica ها (3 از 3)، عملیات نوشتن موفق در نظر گرفته شد.

سپس یک درخواست Strong GET به Replica 2 ارسال شد. از آنجا که Replica 2 نقش Secondary را دارد، درخواست را به Replica 1 به عنوان Primary فوروارد کرد. Replica 1 جدیدترین نسخه داده را بازگرداند و پاسخ از طریق Replica 2 به کلاینت ارسال شد. نتیجه نشان داد که مقدار strong_value با نسخه 1 بازیابی شده است.

این آزمایش نشان می‌دهد که در مدل Strong Consistency، عملیات خواندن از طریق Primary انجام می‌شود و بنابراین احتمال مشاهده داده‌های قدیمی (Stale Read) از بین می‌رود. همچنین لاگ‌های سیستم نشان می‌دهند که مکانیزم Quorum برای عملیات نوشتن و مکانیزم Coordinated Read برای عملیات خواندن با موفقیت پیاده‌سازی شده‌اند.

بخش سوم: نسخه گذاری و حل تعارض

• نگهداری نسخه برای هر کلید

برای هر کلید در سیستم یک فیلد **Version** در ساختار **ValueRecord** نگهداری می‌شود. هنگام اجرای هر عملیات PUT، نسخه جدید برابر با نسخه فعلی به علاوه یک در نظر گرفته می‌شود ($\text{currentRecord.Version} + 1$). این مکانیزم باعث می‌شود تاریخچه منطقی تغییرات هر کلید قابل ردیابی باشد و Replicaها بتوانند جدیدترین نسخه داده را از نسخه‌های قدیمی تشخیص دهند. نسخه‌ها همچنین در فرآیند Replication و تشخیص تعارض مورد استفاده قرار می‌گیرند.

```
type ValueRecord struct {
    Key      string `json:"key"`
    Value     string `json:"value"`
    Version   int    `json:"version"`
    UpdatedBy string `json:"updated_by"`
    Timestamp int64  `json:"timestamp"` // UnixNano for Last-Write-Wins resolution
}
```

• جلوگیری از overwrite شدن مقدار جدید با مقدار قدیم

در زمان دریافت پیام Replication، Replica مقصد نسخه داده دریافتی را با نسخه ذخیره‌شده محلی مقایسه می‌کند. اگر نسخه دریافتی بزرگتر باشد، داده جدید جایگزین داده قبلی می‌شود؛ اما اگر نسخه دریافتی کوچکتر باشد، پیام کاملاً نادیده گرفته می‌شود. این سیاست از بازنویسی شدن داده‌های جدید توسط پیام‌های قدیمی یا تأخیردار شبکه جلوگیری می‌کند و تضمین می‌کند که وضعیت سیستم همواره به سمت جدیدترین نسخه حرکت کند.

• مدیریت Conflict همزمان

در صورتی که دو Replica تقریباً همزمان مقدارهای متفاوتی را برای یک کلید ثبت کنند، ممکن است هر دو نسخه دارای شماره نسخه یکسان باشند. برای مدیریت این وضعیت، سیستم هنگام دریافت دو رکورد با نسخه برابر وارد فرآیند تشخیص تعارض می‌شود. در سناریوی تعارض همزمان، Replicaها پس از تبادل پیام‌های Replication مقدار نهایی یکسانی را انتخاب کرده و در نتیجه همه گره‌ها به وضعیت مشترک و سازگار همگرا می‌شوند.

• توضیح سیاست انتخاب شده

برای حل تعارض‌ها از سیاست Last-Write-Wins استفاده شده است. در این روش رکوردی که دارای Timestamp جدیدتری باشد به عنوان نسخه معتبر انتخاب می‌شود. اگر دو رکورد Timestamp یکسان داشته باشند، شناسه UpdatedBy به عنوان Tie-Breaker استفاده می‌شود تا نتیجه تصمیم‌گیری قطعی و یکسان باشد. این روش ساده، سربار کمی دارد و باعث می‌شود تمامی Replicaها پس از تبادل پیام‌ها روی یک مقدار نهایی توافق کنند.

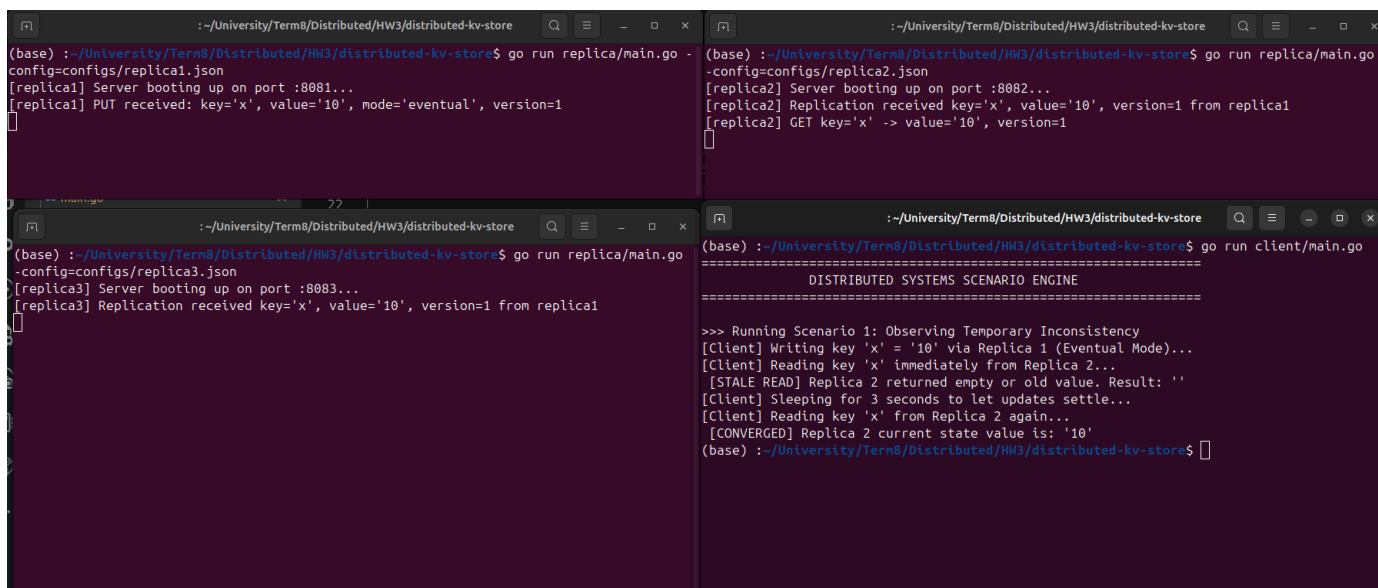
بخش چهارم: آزمایش و تحلیل

سناریوی ۱: مشاهده ناسازگاری موقت

در این آزمایش رفتار مدل Eventual Consistency بررسی شد. ابتدا مقدار $x=10$ روی Replica 1 نوشته شد. بلافاصله پس از عملیات PUT، یک درخواست GET برای همان کلید به Replica 2 ارسال گردید. در این لحظه Replica 2 هنوز پیام تکثیر را دریافت نکرده بود، بنابراین مقدار جدید را در اختیار نداشت و نتیجه خواندن شامل مقدار قدیمی یا خالی بود. این وضعیت نمونه‌ای از Stale Read در سیستم‌های دارای Eventual Consistency است.

پس از گذشت چند ثانیه، Replica 1 پیام تکثیر را برای Replica 2 ارسال کرد و Replica 2 مقدار جدید را دریافت و ذخیره نمود. با انجام مجدد عملیات GET روی Replica 2، مقدار 10 بازگردانده شد. این رفتار نشان می‌دهد که اگرچه همه Replica ها بلافاصله پس از نوشتن دارای داده یکسان نیستند، اما پس از گذشت زمان و تکمیل فرایند تکثیر، همه Replica ها به یک وضعیت مشترک همگرا می‌شوند.

نتیجه این آزمایش نشان می‌دهد که در مدل Eventual Consistency ممکن است برای مدت کوتاهی داده‌های قدیمی مشاهده شوند، اما در نهایت همه Replica ها به آخرین مقدار ثبت‌شده دست پیدا می‌کنند. این موضوع یکی از تفاوت‌های اصلی بین Eventual Consistency و Strong Consistency است.



```
(base) ~/University/Term8/Distributed/HW3/distributed-kv-store$ go run replica/main.go -config=configs/replica1.json
[replica1] Server booting up on port :8081...
[replica1] PUT received: key='x', value='10', mode='eventual', version=1

(base) ~/University/Term8/Distributed/HW3/distributed-kv-store$ go run replica/main.go -config=configs/replica2.json
[replica2] Server booting up on port :8082...
[replica2] Replication received key='x', value='10', version=1 from replica1
[replica2] GET key='x' -> value='10', version=1

(base) ~/University/Term8/Distributed/HW3/distributed-kv-store$ go run replica/main.go -config=configs/replica3.json
[replica3] Server booting up on port :8083...
[replica3] Replication received key='x', value='10', version=1 from replica1

(base) ~/University/Term8/Distributed/HW3/distributed-kv-store$ go run client/main.go
=====
DISTRIBUTED SYSTEMS SCENARIO ENGINE
=====

>>> Running Scenario 1: Observing Temporary Inconsistency
[Client] Writing key 'x' = '10' via Replica 1 (Eventual Mode)...
[Client] Reading key 'x' immediately from Replica 2...
[STALE READ] Replica 2 returned empty or old value. Result: ''
[Client] Sleeping for 3 seconds to let updates settle...
[Client] Reading key 'x' from Replica 2 again...
[CONVERGED] Replica 2 current state value is: '10'
(base) ~/University/Term8/Distributed/HW3/distributed-kv-store$
```

سناریوی ۲: خرابی یک replica

7-2 سناریو خرابی (Replica Failure) (Replica)

در این آزمایش تأثیر خرابی یکی از Replica ها بر عملکرد سیستم در دو مدل Eventual Consistency و Strong Consistency بررسی شد. ابتدا هر سه Replica اجرا شدند و سپس Replica 3 متوقف شد تا خرابی یک گره شبیه‌سازی شود. پس از آن عملیات PUT در هر دو مدل سازگاری انجام شد.

در حالت **Eventual Consistency**، عملیات نوشتن روی Replica 1 انجام شد. با وجود اینکه Replica 3 از دسترس خارج شده بود، عملیات PUT با موفقیت انجام شد و مقدار جدید برای Replica 2 ارسال گردید. از آنجا که این مدل نیازی به دریافت تاییدیه از سایر Replica ها ندارد، خرابی Replica 3 تأثیری بر موفقیت عملیات نداشت. بنابراین سیستم همچنان در دسترس باقی ماند و عملیات نوشتن بدون انتظار برای پاسخ سایر Replica ها تکمیل شد.

```

[base] :~/University/Term8/Distributed/HW3/distributed-kv-store$ go run replica/main.go -
config=configs/replica1.json
[replica1] Server booting up on port :8081...
[replica1] PUT received: key='y', value='eventual_val', mode='eventual', version=1
[replica1] PUT received: key='y', value='strong_val', mode='strong', version=2
[replica1] Replication ACK received from http://localhost:8082 for key='y'
[replica1] Eventual replication sent key='y' to http://localhost:8082
[replica1] Replication ACK received from http://localhost:8082 for key='y'
[replica1] STRONG WRITE SUCCESS key='y' quorum=2/3
[base] :~/University/Term8/Distributed/HW3/distributed-kv-store$

[base] :~/University/Term8/Distributed/HW3/distributed-kv-store$ go run replica/main.go
-config=configs/replica2.json
[replica2] Server booting up on port :8082...
[replica2] Replication received key='y', value='eventual_val', version=1 from replica1
[replica2] Replication received key='y', value='strong_val', version=2 from replica1
[base] :~/University/Term8/Distributed/HW3/distributed-kv-store$

[base] :~/University/Term8/Distributed/HW3/distributed-kv-store$ go run replica/main.go
-config=configs/replica3.json
[replica3] Server booting up on port :8083...
^Csignal: interrupt
[base] :~/University/Term8/Distributed/HW3/distributed-kv-store$

[base] :~/University/Term8/Distributed/HW3/distributed-kv-store$ go run client/main.go
=====
DISTRIBUTED SYSTEMS SCENARIO ENGINE
=====

>>> Running Scenario 2: Replica Failure
[Simulation] Note: Replica 3 will be treated as failed (simulated by missing responses).
[Client] Testing Eventual consistency with Replica 3 down...
-> Eventual PUT Status: Success=true, Time=510.995µs
[Client] Testing Strong consistency with Replica 3 down...
-> Strong PUT Status: Success=true, Replicas Synced=2, Time=1.94976ms
[base] :~/University/Term8/Distributed/HW3/distributed-kv-store$
```

در حالت **Strong Consistency**، عملیات PUT تنها زمانی موفق در نظر گرفته می‌شود که اکثریت Replicas ها مقدار جدید را تایید کنند. با وجود خرابی Replica 1، Replica 2 و Replica 3 در دسترس بودند و شرط اکثریت (2 از 3 Replica) برقرار شد. همان‌طور که در خروجی مشاهده می‌شود، Replica 1 تأییدیه Replica 2 را دریافت کرده و عملیات با پیام **STRONG WRITE SUCCESS quorum=2/3** با موفقیت پایان یافته است.

```

[base] :~/University/Term8/Distributed/HW3/distributed-kv-store$ go run replica/main.go -
config=configs/replica1.json
[replica1] Server booting up on port :8081...
[replica1] PUT received: key='y', value='eventual_val', mode='eventual', version=1
[replica1] PUT received: key='y', value='strong_val', mode='strong', version=2
[replica1] Replication ACK received from http://localhost:8082 for key='y'
[replica1] STRONG WRITE SUCCESS key='y' quorum=2/3
[replica1] Replication ACK received from http://localhost:8082 for key='y'
[replica1] Eventual replication sent key='y' to http://localhost:8082
[base] :~/University/Term8/Distributed/HW3/distributed-kv-store$

[base] :~/University/Term8/Distributed/HW3/distributed-kv-store$ go run replica/main.go
-config=configs/replica2.json
[replica2] Server booting up on port :8082...
[replica2] Replication received key='y', value='strong_val', version=2 from replica1
[replica2] Replication received key='y', value='eventual_val', version=1 from replica1
[base] :~/University/Term8/Distributed/HW3/distributed-kv-store$

[base] :~/University/Term8/Distributed/HW3/distributed-kv-store$ go run replica/main.go
-config=configs/replica3.json
[replica3] Server booting up on port :8083...
^Csignal: interrupt
[base] :~/University/Term8/Distributed/HW3/distributed-kv-store$

[base] :~/University/Term8/Distributed/HW3/distributed-kv-store$ go run client/main.go
=====
DISTRIBUTED SYSTEMS SCENARIO ENGINE
=====

>>> Running Scenario 2: Replica Failure
[Simulation] Note: Replica 3 will be treated as failed (simulated by missing responses).
[Client] Testing Eventual consistency with Replica 3 down...
-> Eventual PUT Status: Success=true, Time=493.41µs
[Client] Testing Strong consistency with Replica 3 down...
-> Strong PUT Status: Success=true, Replicas Synced=2, Time=1.367504ms
[base] :~/University/Term8/Distributed/HW3/distributed-kv-store$
```

نتایج این آزمایش نشان می‌دهد که در مدل Eventual Consistency، دسترس‌پذیری سیستم بیشتر است و خرابی یک Replica مانع انجام عملیات نوشتن نمی‌شود. در مقابل، مدل Strong Consistency برای حفظ سازگاری داده‌ها نیازمند تایید اکثریت Replicas ها است. با این حال، تا زمانی که حداقل دو Replica از سه Replica فعال باشند، سیستم همچنان قادر به پذیرش عملیات نوشتن خواهد بود. در صورت خرابی دو Replica به طور همزمان، شرط اکثریت برقرار نشده و عملیات Strong PUT با شکست مواجه می‌شود.

مدل سازگاری	وضعیت Replica 3	نتیجه PUT	تعداد Replica های به‌روزرسانی‌شده
Eventual Consistency	از کار افتاده	موفق	Replica 1 and Replica 2
Strong Consistency	از کار افتاده	موفق	Replica 1 and Replica 2 (Quorum=2/3)

این سناریو نشان می‌دهد که Eventual Consistency دسترس‌پذیری بیشتری فراهم می‌کند، در حالی که Strong Consistency برای حفظ سازگاری داده‌ها به وجود اکثریت Replicaها وابسته است.

سناریوی ۳: تعارض همزمان

در این آزمایش، برای کلید مشترک **conflict_key** به صورت تقریباً همزمان دو مقدار متفاوت روی دو Replica مختلف نوشته شد. Replica 1 مقدار **apple** و Replica 2 مقدار **banana** را با نسخه یکسان (Version=1) ثبت کردند. در نتیجه، پس از تبادل پیام‌های تکثیر بین Replicaها، دو مقدار متفاوت با نسخه یکسان برای یک کلید در سیستم وجود داشت که منجر به ایجاد تعارض (Conflict) شد.

پس از دریافت پیام‌های تکثیر، هر Replica سیاست حل تعارض پیاده‌سازی شده را اجرا کرد. در این پروژه از روش **Last-Write-Wins** استفاده شده است. در این روش، زمانی که دو مقدار دارای نسخه یکسان باشند، مقدار دارای Timestamp جدیدتر انتخاب می‌شود. در صورت برابر بودن Timestamp نیز شناسه Replica به عنوان معیار نهایی تصمیم‌گیری استفاده می‌شود.

مطابق خروجی سیستم، مقدار **apple** به عنوان برنده تعارض انتخاب شد و تمامی Replicaها پس از تبادل پیام‌ها به همان مقدار همگرا شدند. در پایان آزمایش، هر سه Replica مقدار **apple** را برای کلید **conflict_key** گزارش کردند که نشان‌دهنده موفقیت مکانیزم حل تعارض و همگرایی نهایی سیستم است.

نتیجه نهایی

Replica مقدار نهایی

apple Replica 1

apple Replica 2

apple Replica 3

این آزمایش نشان می‌دهد که حتی در صورت وقوع نوشتن‌های همزمان و ایجاد ناسازگاری موقت، سیاست حل تعارض انتخاب‌شده قادر است تمام Replicaها را به یک وضعیت نهایی و سازگار هدایت کند.


```

[base] :~/University/Term8/Distributed/HW3/distributed-kv-store$ go run replica/main.go -
-config=configs/replica1.json
[replica1] Server booting up on port :8081...
[replica1] PUT received: key='conflict_key', value='apple', mode='eventual', version=1
[replica1] Replication received key='conflict_key', value='banana', version=1 from replic
a2
[replica1] Replication ACK received from http://localhost:8083 for key='conflict_key'
[replica1] Eventual replication sent key='conflict_key' to http://localhost:8083
[replica1] Replication ACK received from http://localhost:8082 for key='conflict_key'
[replica1] Eventual replication sent key='conflict_key' to http://localhost:8082
[replica1] GET key='conflict_key' -> value='apple', version=1
[base] :~/University/Term8/Distributed/HW3/distributed-kv-store$ go run replica/main.go
-config=configs/replica2.json
[replica2] Server booting up on port :8082...
[replica2] PUT received: key='conflict_key', value='banana', mode='eventual', version=1
[replica2] Replication received key='conflict_key', value='apple', version=1 from replic
a1
[replica2] Replication ACK received from http://localhost:8083 for key='conflict_key'
[replica2] Eventual replication sent key='conflict_key' to http://localhost:8083
[replica2] Conflict resolved (LWM): key='conflict_key' chose value='apple'
[replica2] Replication ACK received from http://localhost:8081 for key='conflict_key'
[replica2] Eventual replication sent key='conflict_key' to http://localhost:8081
[replica2] GET key='conflict_key' -> value='apple', version=1
[base] :~/University/Term8/Distributed/HW3/distributed-kv-store$ go run replica/main.go
-config=configs/replica3.json
[replica3] Server booting up on port :8083...
[replica3] Replication received key='conflict_key', value='banana', version=1 from repli
ca2
[replica3] Replication received key='conflict_key', value='apple', version=1 from replic
a1
[replica3] Conflict resolved (LWM): key='conflict_key' chose value='apple'
[replica3] GET key='conflict_key' -> value='apple', version=1
[base] :~/University/Term8/Distributed/HW3/distributed-kv-store$ go run client/main.go
=====
DISTRIBUTED SYSTEMS SCENARIO ENGINE
=====
>>> Running Scenario 3: Concurrent Conflict
[Client] Racing concurrent writes across different replication endpoints...
[Client] Waiting 2 seconds for conflict resolution cycles to settle...
-> Final states: Replica1='apple', Replica2='apple', Replica3='apple'
[SUCCESS] Cluster reached uniform agreement via conflict resolution strategy.
(base) :~/University/Term8/Distributed/HW3/distributed-kv-store$
```

سناریوی ۴: تأثیر تأخیر شبکه

سناریوی ۴: تأثیر تأخیر شبکه بر همگرایی Replica ها (Eventual Consistency)

در این آزمایش تأخیر مصنوعی شبکه برای پیام‌های Replication به ترتیب برابر با ۵۰۰، ۱۰۰ و ۲۰۰۰ میلی‌ثانیه تنظیم شد. سپس یک عملیات PUT روی 1 Replica انجام شد و زمان لازم برای همگرا شدن تمام Replica ها اندازه‌گیری گردید. همچنین تعداد دفعاتی که در طول فرآیند همگرایی مقدار قدیمی (Stale Read) مشاهده شد ثبت شد.

نتایج نشان داد که زمان پاسخ PUT تقریباً ثابت و بسیار کم باقی می‌ماند، زیرا در مدل Eventual Consistency پس از ثبت محلی داده، پاسخ بلافاصله به کاربر بازگردانده می‌شود و Replica ها در پس‌زمینه به‌روزرسانی می‌شوند. با افزایش تأخیر شبکه، زمان همگرایی Replica ها افزایش یافت؛ به طوری که برای تأخیرهای ۵۰۰، ۱۰۰ و ۲۰۰۰ میلی‌ثانیه به ترتیب حدود ۵۴ میلی‌ثانیه، ۵۳۶ میلی‌ثانیه و ۲,۰۴ ثانیه اندازه‌گیری شد.

همچنین تعداد Stale Read ها با افزایش تأخیر بیشتر شد. این موضوع نشان می‌دهد که در مدل Eventual Consistency هرچه انتقال پیام‌های Replication کندتر باشد، بازه زمانی ناسازگاری میان Replica ها طولانی‌تر خواهد بود و احتمال مشاهده داده قدیمی توسط کاربران افزایش می‌یابد. با این حال پس از گذشت زمان کافی، همه Replica ها به یک وضعیت یکسان همگرا شدند.

```
(base) :~/University/Term8/Distributed/HW3/distributed-kv-store$ go run replica/main.go -config=configs/replica1.json
[replica1] Server booting up on port :8081...
[replica1] PUT received: key='eventual_key_0', value='eventual_val_0', mode='eventual', version=1
[replica1] Replication ACK received from http://localhost:8082 for key='eventual_key_0'
[replica1] Eventual replication sent key='eventual_key_0' to http://localhost:8082
[replica1] Replication ACK received from http://localhost:8083 for key='eventual_key_0'
[replica1] Eventual replication sent key='eventual_key_0' to http://localhost:8083
[replica1] PUT received: key='eventual_key_500', value='eventual_val_500', mode='eventual', version=1
[replica1] Replication ACK received from http://localhost:8082 for key='eventual_key_500'
[replica1] Eventual replication sent key='eventual_key_500' to http://localhost:8082
[replica1] Replication ACK received from http://localhost:8083 for key='eventual_key_500'
[replica1] Eventual replication sent key='eventual_key_500' to http://localhost:8083
[replica1] PUT received: key='eventual_key_2000', value='eventual_val_2000', mode='eventual', version=1
[]

(base) :~/University/Term8/Distributed/HW3/distributed-kv-store$ go run replica/main.go -config=configs/replica2.json
[replica2] Server booting up on port :8082...
[replica2] Replication received key='eventual_key_0', value='eventual_val_0', version=1 from replica1
[replica2] GET keys='eventual_key_0' -> value='eventual_val_0', version=1
[replica2] Replication received key='eventual_key_500', value='eventual_val_500', version=1 from replica1
[replica2] GET keys='eventual_key_500' -> value='eventual_val_500', version=1
[replica2] Replication received key='eventual_key_2000', value='eventual_val_2000', version=1 from replica1
[replica2] GET keys='eventual_key_2000' -> value='eventual_val_2000', version=1
[]

(base) :~/University/Term8/Distributed/HW3/distributed-kv-store$ go run replica/main.go -config=configs/replica3.json
[replica3] Server booting up on port :8083...
[replica3] Replication received keys='eventual_key_0', value='eventual_val_0', version=1 from replica1
[replica3] GET key='eventual_key_0' -> value='eventual_val_0', version=1
[replica3] Replication received keys='eventual_key_500', value='eventual_val_500', version=1 from replica1
[replica3] GET key='eventual_key_500' -> value='eventual_val_500', version=1
[replica3] Replication received keys='eventual_key_2000', value='eventual_val_2000', version=1 from replica1
[replica3] GET key='eventual_key_2000' -> value='eventual_val_2000', version=1
[]

(base) :~/University/Term8/Distributed/HW3/distributed-kv-store$ go run client/main.go
=====
DISTRIBUTED SYSTEMS SCENARIO ENGINE
=====
>>> Running Scenario 4: Effect of Network Latency (eventual)
ir--- Testing Latency Configuration: 0ms ---
ar[eventual] PUT Latency=601.719µs | Convergence=54.052093ms | Stale Reads=1 | Replicas Updated=1
ir--- Testing Latency Configuration: 500ms ---
st[eventual] PUT Latency=465.258µs | Convergence=536.258485ms | Stale Reads=10 | Replicas Updated=1
s
--- Testing Latency Configuration: 2000ms ---
/d[eventual] PUT Latency=543.833µs | Convergence=2.043424636s | Stale Reads=38 | Replicas Updated=1
k-
le(base) :~/University/Term8/Distributed/HW3/distributed-kv-store$ []
```

سناریوی ۴: تأثیر تأخیر شبکه در مدل Strong Consistency

در این آزمایش تأخیر شبکه برای پیام‌های Replication به مقادیر ۰، ۵۰۰ و ۲۰۰۰ میلی‌ثانیه تنظیم شد. سپس عملیات PUT در حالت Strong Consistency اجرا گردید. در این مدل، عملیات نوشتن تنها زمانی موفق محسوب می‌شود که اکثریت Replica ها دریافت و ثبت داده را تایید کنند.

نتایج نشان داد که با افزایش تأخیر شبکه، زمان پاسخ PUT نیز افزایش می‌یابد. در تأخیر ۰ میلی‌ثانیه زمان پاسخ حدود ۲ میلی‌ثانیه و در تأخیر ۵۰۰ میلی‌ثانیه حدود ۵۰۲ میلی‌ثانیه بود. برخلاف مدل Eventual Consistency، در این حالت هیچ Stale Read مشاهده نشد، زیرا سیستم پیش از بازگرداندن پاسخ موفق، منتظر تأیید Replica ها می‌ماند.

در آزمایش با تأخیر ۲۰۰۰ میلی‌ثانیه عملیات PUT ناموفق شد و حد نصاب (Quorum) لازم به دست نیامد. علت این موضوع برابر بودن تأخیر شبکه با زمان Timeout ارتباطات بود که باعث شد Replica ها نتوانند در زمان مقرر پاسخ دهند. این نتیجه نشان می‌دهد که در مدل Strong Consistency افزایش تأخیر شبکه مستقیماً بر زمان پاسخ و حتی موفقیت عملیات نوشتن اثر می‌گذارد.

مقایسه نتایج نشان می‌دهد که Eventual Consistency پاسخ سریع‌تر ولی احتمال مشاهده داده قدیمی را فراهم می‌کند، در حالی که Strong Consistency با افزایش زمان پاسخ، از مشاهده مقادیر ناسازگار جلوگیری می‌کند.

```
~/University/Term8/Distributed/HW3/distributed-kv-store$ go run replica/main.go -config=configs/replica1.json
[replica1] Server booting up on port :8081...
[replica1] PUT received: key='strong_key_0', value='strong_val_0', mode='strong', version=1
[replica1] Replication ACK received from http://localhost:8082 for key='strong_key_0'
[replica1] Replication ACK received from http://localhost:8083 for key='strong_key_0'
[replica1] STRONG WRITE SUCCESS key='strong_key_0' quorum=3/3
[replica1] PUT received: key='strong_key_500', value='strong_val_500', mode='strong', version=1
[replica1] Replication ACK received from http://localhost:8082 for key='strong_key_500'
[replica1] Replication ACK received from http://localhost:8083 for key='strong_key_500'
[replica1] STRONG WRITE SUCCESS key='strong_key_500' quorum=3/3
[replica1] PUT received: key='strong_key_2000', value='strong_val_2000', mode='strong', version=1
[replica1] STRONG WRITE FAILED key='strong_key_2000' quorum=1/3

~/University/Term8/Distributed/HW3/distributed-kv-store$ go run replica/main.go -config=configs/replica2.json
[replica2] Server booting up on port :8082...
[replica2] Replication received key='strong_key_0', value='strong_val_0', version=1 from replica1
[replica2] GET key='strong_key_0' -> value='strong_val_0', version=1
[replica2] Replication received key='strong_key_500', value='strong_val_500', version=1 from replica1
[replica2] GET key='strong_key_500' -> value='strong_val_500', version=1
[replica2] Replication received key='strong_key_2000', value='strong_val_2000', version=1 from replica1

~/University/Term8/Distributed/HW3/distributed-kv-store$ go run replica/main.go -config=configs/replica3.json
[replica3] Server booting up on port :8083...
[replica3] Replication received key='strong_key_0', value='strong_val_0', version=1 from replica1
[replica3] GET key='strong_key_0' -> value='strong_val_0', version=1
[replica3] Replication received key='strong_key_500', value='strong_val_500', version=1 from replica1
[replica3] GET key='strong_key_500' -> value='strong_val_500', version=1
[replica3] Replication received key='strong_key_2000', value='strong_val_2000', version=1 from replica1

~/University/Term8/Distributed/HW3/distributed-kv-store$ go run client/main.go
=====
DISTRIBUTED SYSTEMS SCENARIO ENGINE
=====
>>> Running Scenario 4: Effect of Network Latency (strong)
in --- Testing Latency Configuration: 0ms ---
ap [strong] PUT Latency=1.977702ms | Convergence=517.78µs | Stale Reads=0 | Replicas Updated=3
in --- Testing Latency Configuration: 500ms ---
st [strong] PUT Latency=501.933084ms | Convergence=1.038681ms | Stale Reads=0 | Replicas Updated=3
s
d --- Testing Latency Configuration: 2000ms ---
/d [strong] PUT FAILED
k- (base) ~/University/Term8/Distributed/HW3/distributed-kv-store$
le
```

مقایسه خروجی ها نشان می دهد که انتخاب بین strong consistency و eventual consistency یک trade-off مستقیم بین performance و داده های به روز است. حالت eventual consistency برای system هایی که نیازمند high availability و تاخیر بسیار پایین هستند ایده آل است، اما client باید احتمال stale read را مدیریت کند. در مقابل، strong consistency یکپارچگی داده ها را تضمین می کند، اما در network هایی با latency بالا باعث افت شدید performance شده و در صورت node failure برای اکثریت اعضا، قابلیت write را از دست می دهد.